The Journal of Open Source Software

NeoN: A domain specific language for computational fluid dynamics

Dheeraj Raghunathan [1], **Chih-Ta Wang**[1], **Hendrik Hetmann**[2], **Bevan Jones**[3], **Gregor Weiss**[4], **Marcel Koch**[5], **Yu-Hsiang Tsai** [1], **Henning Scheufler**[3], **Gregor Olenik** [1], **and Hartwig Anzt** [1,6]

1 Technical University of Munich, Germany **2** Upstream CFD GmbH, Germany **3** Independent Researcher, Germany **4** High-Performance Computing Center Stuttgart (HLRS), Germany **5** Karlsruhe Institute of Technology, Germany **6** Innovative Computing Laboratory, University of Tennessee, United States

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Rohit Goswami

Reviewers:

- @streeve
- @XzzX
- @lindsayad

Submitted: 29 April 2026
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

NeoN is an open-source C++20-compliant library for developing high-performance computational fluid dynamics (CFD) software on heterogeneous GPU and multicore CPU systems. It provides a domain specific language (DSL) for expressing partial differential equations through finite-volume operators, while retaining fine-grained control over discretisation, matrix assembly and execution. Discretisation policies generate stencil coefficients, which are assembled into standard sparse matrix formats (e.g., CSR) and solved using external linear solver backends such as Ginkgo. NeoN utilises lazy evaluation to defer matrix assembly until execution. This architecture avoids unnecessary temporaries during matrix assembly and enables optimisations on the abstract syntax tree (AST), for example via fused GPU kernels with higher arithmetic intensity. Additionally, NeoN is designed for easy integration with other CFD frameworks through its powerful API. To ensure performance across architectures while guaranteeing correct results, NeoN employs a robust continuous integration system that includes automated build validation, extensive GPU compatibility testing on AMD, NVIDIA, and Intel GPUs, and automated benchmarking.

Statement of Need

The finite volume method (FVM) is a widely used discretisation approach in academic and industrial CFD applications due to its exact local conservation, robustness and flexibility. However, many of the widely used CFD frameworks were originally designed for single-core or moderately parallel CPU architectures. Since the development of classical open-source FVM codes like OpenFOAM (Weller et al., 1998) and code_saturne (Archambeau et al., 2004), computer hardware has evolved significantly: floating-point operations have become relatively inexpensive, while data movement has emerged as the primary performance bottleneck. Modern high-performance computing (HPC) platforms are inherently parallel, with hierarchical memory systems and accelerator-centric architectures, where performance is often limited by memory bandwidth rather than raw compute capability - an issue particularly relevant for CFD workflows, which are typically memory-bound with low arithmetic intensity. Accelerator-based architectures are increasingly adopted in modern HPC systems for scientific computing and AI, but legacy CFD codes often rely on backend-specific adaptations or refactoring, which complicate performance optimisation on such architectures (Olenik et al., 2024).

NeoN targets this gap by providing a versatile performance-portable CFD core that seamlessly integrates GPU-native data structures and finite-volume operator abstractions within a unified programming model. Using Kokkos (Trott et al., 2022) to implement a platform-portable

DRAFT

parallelisation layer and a domain-specific interface for discretisation operators, NeoN enables the creation of modern CFD solvers, as demonstrated in NeoFOAM (NeoFOAM developers, 2026). These solvers can efficiently target a variety of CPU-GPU architectures without being restricted to a single hardware backend or requiring duplicate numerical implementations. Additionally, through its companion project NeoFOAM, NeoN facilitates progressive modernisation of legacy solvers, including OpenFOAM-based codes, with minimal refactoring.

State of the Field

Over the past two decades, significant efforts have been devoted to developing an abstraction mechanism and domain-specific representations for FVM codes. Early CFD frameworks, such as OpenFOAM, introduced object-oriented tensorial abstractions (Weller et al., 1998) in C++, enabling finite-volume discretisations to be expressed in a compact and expressive form that has seen widespread industrial adoption. Despite their success, these abstractions are typically integrated within solver-specific infrastructures, making it difficult to reuse finite-volume components or achieve interoperability between independently developed CFD frameworks. Embedded DSLs like Life (Prud'homme, 2007) demonstrate improved expressiveness of numerical formulations, yet struggle with solver-agnostic semantics and performance portability. In contrast, external DSLs and code-generation frameworks - such as ExaSlang (Kuckuk & Köstler, 2016), ExaStencils (Lengauer et al., 2014) and Dawn (Osuna et al., 2020) have enabled aggressive optimisation of stencil-based operators (Kuckuk et al., 2017). However, they are restricted to structured discretisations and predefined numerical patterns, and are not designed to support the unstructured FV methods prevalent in general-purpose and industrial CFD. Recent efforts, such as Finch (Heisler et al., 2022) and ProtoX (Mankad et al., 2024), offer high performance portability, although they remain closely tied to specific solver frameworks, grid structures, or code-generation pipelines. The existing works demonstrate that core finite-volume concepts - such as flux evaluation, control-volume integration, and discrete operators - can be expressed at a high level and optimised effectively when decoupled. Yet, a persistent gap remains: the lack of a modular CFD core that integrates high-level finite-volume abstractions with performance-portable execution on heterogeneous CPU-GPU architectures. NeoN aims to address this gap by combining the usability of embedded DSLs with the optimisation potential of external DSL approaches through a high-level finite-volume operator interface with lazy evaluation and performance-portable execution on heterogeneous architectures.

Software Design

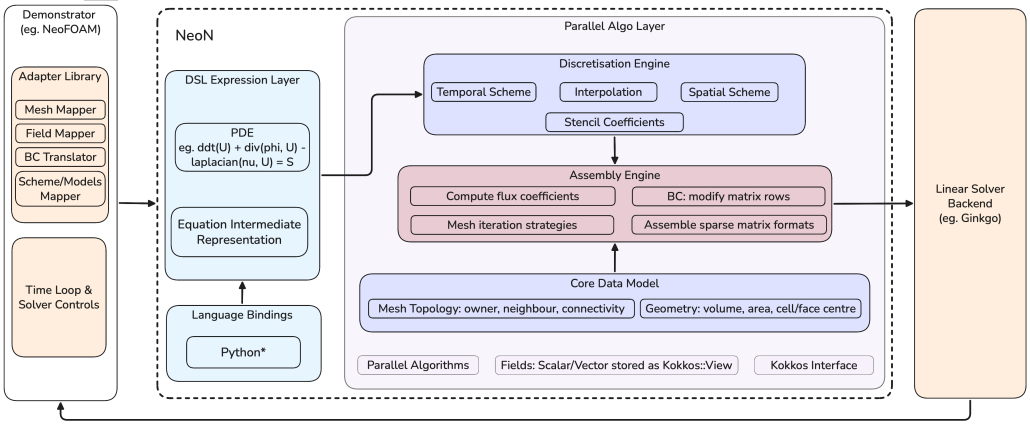


Figure 1: Design overview of the NeoN software architecture.

DRAFT

NeoN is built around a layered, modular architecture as illustrated in Figure 1. Its design cleanly separates the distinct stages: DSL-based equation representation, discretisation, matrix assembly, and linear solver. This separation allows each layer to evolve independently, making the framework highly extensible and maintainable. At the heart of NeoN is a C++ embedded DSL that allows users to express partial differential equations in a form close to their mathematical representation, while the corresponding finite-volume discretisation is performed internally by the framework. Differential operators like `ddt`, `div`, `laplacian`, or `grad` can be composed directly into equation expressions, for example:

```
auto expr = dsl::imp::ddt(U) + dsl::imp::div(phi, U) - dsl::imp::laplacian(nu, U);
```

Listing 1: Example expression with multiple differential operators.

Rather than evaluating the operators of the expressions eagerly, NeoN employs lazy evaluation to build an intermediate representation that is later interpreted by the discretisation engine. This approach avoids unnecessary temporary allocations and reduces memory traffic. Thus, the expression shown in Listing 1 is only fully assembled when an operation like `solve(expr, …)` requires the assembly. The discretisation engine then applies selected temporal and spatial schemes to generate local stencil contributions for the finite-volume formulation. The core data model encapsulates mesh topology, geometric metrics, and scalar or vector fields in performance-portable memory abstractions. These contributions are then forwarded to an assembly engine that constructs linear system of equations in common sparse matrix storage formats such as CSR and COO, rather than solver-specific representations. This design choice improves interoperability with external solver libraries. NeoN integrates Ginkgo (Anzt et al., 2022; Ginkgo developers, 2026) to solve the resulting sparse linear systems, providing access to portable and scalable iterative solvers and preconditioners. High-level workflow concerns such as time loops, solver control, and I/O are intentionally handled outside the core library by external applications. The `Demonstrator` (NeoFOAM) illustrates how NeoN can be integrated into OpenFOAM-style workflows. In addition, a language-binding layer enables the use of NeoN's DSL from other languages such as Python.

Performance Evaluation

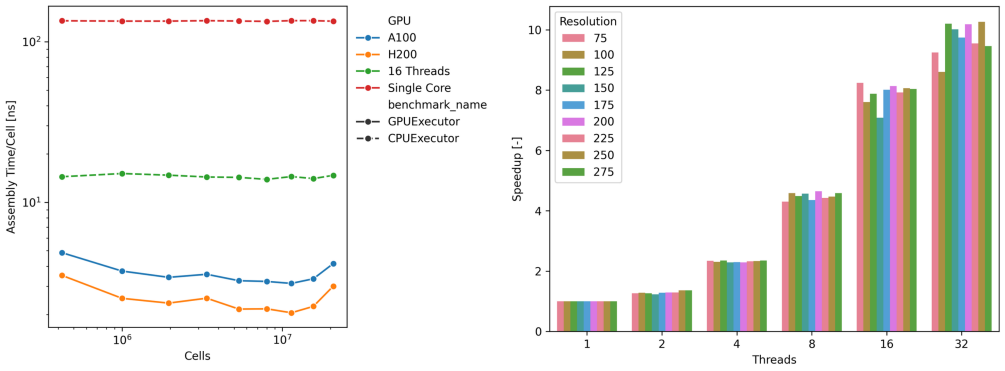


Figure 2: Computational performance evaluating a simplified momentum transport equation. Left: required assembly time per cell for different grid resolutions on a single core, using 16 threads, and an A100 and H200 NVIDIA GPU. Right: speed-up relative to a single core for different numbers of threads across various grid resolutions.

To evaluate the performance of NeoN, the wall-clock time required for the matrix assembly of the expression defined in Listing 1 is measured across multiple hardware backends and problem sizes. The computational domain consists of a unit cube discretised using a uniform hexahedral

DRAFT

mesh with varying resolutions. Benchmarks are conducted on representative high-performance computing hardware, including an AMD EPYC 9555 processor (64 cores) and NVIDIA A100 and H200 GPUs. Figure 2(Left) reports the assembly time per cell for different grid resolutions, comparing single-core CPU execution, multi-core CPU execution (16 threads), and GPU execution on an A100 and H200 device. The results indicate that GPU acceleration yields a reduction in assembly time per cell by approximately a factor of 20 to 50 relative to single-core CPU execution. In addition, strong scaling experiments using the OpenMP backend on up to 32 CPU cores are presented in Figure 2(Right). A speedup of approximately 10x is achieved when scaling from one to 32 threads. Beyond this point, no further performance gains are observed, which is attributed to memory bandwidth limitations leading to increased contention among threads.

Research Impact Statement

NeoN has documented use in the EXASIM research project (Olenik & Anzt, 2026), where it is presented as an approach for GPU-based assembly of finite-volume operators on heterogeneous CPU/GPU systems. This demonstrates its use in active CFD research workflows.

AI Usage Disclosure

The manuscript was prepared by the authors and refined with AI assistance (GPT-5.4) to enhance clarity, conciseness, and grammatical correctness. The authors did not intentionally use generative AI tools in software development. As NeoN is an open-source project with multiple contributors, the authors cannot ascertain whether any contributed code was AI-assisted. Nevertheless, all code described in this paper was reviewed by human maintainers. The authors have carefully verified all technical content and take full responsibility for the manuscript and the software it describes.

Acknowledgements

Parts of this work were supported by the German Federal Ministry of Education and Research (grant number 16ME0676K).

References

Anzt, H., Cojean, T., Flegar, G., Göbel, F., Grützmacher, T., Nayak, P., Ribizel, T., Tsai, Y. M., & Quintana-Ortí, E. S. (2022). Ginkgo: A Modern Linear Operator Algebra Framework for High Performance Computing. *ACM Transactions on Mathematical Software*, *48*(1), 1–33. https://doi.org/10.1145/3480935

Archambeau, F., Méchitoua, N., & Sakiz, M. (2004). Code Saturne: A Finite Volume Code for the computation of turbulent incompressible flows - Industrial Applications. *International Journal on Finite Volumes*, *1*(1). https://hal.science/hal-01115371

Ginkgo developers. (2026). *Ginkgo*. https://github.com/ginkgo-project/ginkgo.git

Heisler, E., Deshmukh, A., & Sundar, H. (2022). Finch: Domain Specific Language and Code Generation for Finite Element and Finite Volume in Julia. In D. Groen, C. de Mulatier, M. Paszynski, V. V. Krzhizhanovskaya, J. J. Dongarra, & P. M. A. Sloot (Eds.), *Computational Science - ICCS 2022* (pp. 118–132). Springer International Publishing. https://doi.org/10.1007/978-3-031-08751-6_9

Kuckuk, S., Haase, G., Vasco, D. A., & Köstler, H. (2017). Towards generating efficient flow solvers with the ExaStencils approach. *Concurrency and Computation: Practice and*

DRAFT

Experience, *29*(17). https://doi.org/10.1002/cpe.4062

Kuckuk, S., & Köstler, H. (2016). Automatic Generation of Massively Parallel Codes from ExaSlang. *Computation*, *4*(3), 27. https://doi.org/10.3390/computation4030027

Lengauer, C., Apel, S., Bolten, M., Größlinger, A., Hannig, F., Köstler, H., Rüde, U., Teich, J., Grebhahn, A., Kronawitter, S., Kuckuk, S., Rittich, H., & Schmitt, C. (2014). ExaStencils: Advanced Stencil-Code Engineering. In *Euro-Par 2014: Parallel Processing Workshops* (Vol. 8806, pp. 553–564). Springer International Publishing. https://doi.org/10.1007/978-3-319-14313-2_47

Mankad, H., Haque Monil, M. A., Rao, S., Colella, P., Van Straalen, B., Franchetti, F., & Vetter, J. S. (2024). A Performance-Portable MultiGPU Implementation of 3D Euler Equations using ProtoX and IRIS. *SC24-W: Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 1723–1731. https://doi.org/10.1109/SCW63240.2024.00215

NeoFOAM developers. (2026). *NeoFOAM*. https://github.com/exasim-project/NeoFOAM

Olenik, G., & Anzt, H. (2026). *Exascale-fähige softwarewerkzeuge zur strömungssimulation im industriellen design- und optimierungsprozess - EXASIM : Richtlinie: Neue methods und technologien für das exascale-höchstleistungsrechnen (SCALEXA)*. https://doi.org/10.34657/31648

Olenik, G., Koch, M., Boutanios, Z., & Anzt, H. (2024). Towards a platform-portable linear algebra backend for OpenFOAM. *Meccanica*, *60*(6), 1659–1672. https://doi.org/10.1007/s11012-024-01806-1

Osuna, C., Wicky, T., Thuering, F., Hoefler, T., & Fuhrer, O. (2020). Dawn: A High-level Domain-Specific Language Compiler Toolchain for Weather and Climate Applications. *Supercomputing Frontiers and Innovations*, *7*(2), 79–97. https://doi.org/10.14529/jsfi200205

Prud'homme, C. (2007). Life: Overview of a Unified C++ Implementation of the Finite and Spectral Element Methods in 1D, 2D and 3D. In B. Kågström, E. Elmroth, J. Dongarra, & J. Waśniewski (Eds.), *Applied Parallel Computing. State of the Art in Scientific Computing* (pp. 712–721). Springer Berlin Heidelberg. https://doi.org/10.1007/978-3-540-75755-9_87

Trott, C. R., Lebrun-Grandie, D., Arndt, D., Ciesko, J., Dang, V., Ellingwood, N., Gayatri, R., Harvey, E., Hollman, D. S., Ibanez, D., Liber, N., Madsen, J., Miles, J., Poliakoff, D., Powell, A., Rajamanickam, S., Simberg, M., Sunderland, D., Turcksin, B., & Wilke, J. (2022). Kokkos 3: Programming Model Extensions for the Exascale Era. *IEEE Transactions on Parallel and Distributed Systems*, *33*(4), 805–817. https://doi.org/10.1109/TPDS.2021.3097283

Weller, H. G., Tabor, G., Jasak, H., & Fureby, C. (1998). A tensorial approach to computational continuum mechanics using object-oriented techniques. *Computers in Physics*, *12*(6), 620–631. https://doi.org/10.1063/1.168744

DRAFT